

CODING WITH AI: ADVANCED

David Kreutzer & Karsten Silz 30. Juni 2026

Skills und Subagenten Bauen

Subagents

- KI-Spezialisten
 - Analyst, Front-End-Entwickler, Back-End-Entwickler, Tester, Admin, .. 2. Teil der Harness nicht der LLM
- Gleiche LLM
 - Eher Personas
 - Konfiguration direkt im Subagent

ba-writer	admin
db-coder	ui-designer
be-coder	be-test-coder
fe-coder	fe-test-coder

- ba: Business Analyst
- be: Back End
- fe: Front End
- Schneller besseren Code
 - Eigenes Context Window
 - Laufen parallel
 - Regeln anpassbar
 - Mehr Regeln als in CLAUDE.md
 - Lernen
 - Erzeugt manchmal Dateien in .claude/memory/
 - Reviewer Subagents

ba-writer	admin	ba-reviewer	
db-coder	ui-designer	db-reviewer	ui-reviewer
be-coder	be-test-coder	be-reviewer	be-test-reviewer
fe-coder	be-test-coder	fe-reviewer	fe-test-reviewer

- Reviewer Subagents haben eigenes Context Window und sehen Code zum ersten Mal
- Nichtdeterministisch: Wählen vielleicht anderen Ansatz als Coder-Subagents
- Im Ergebnis finden sie mehr Fehler als Coding Agent allein

- Erstellt in Claude
- Direkt editieren
 - /agents, dann "New Agent"
- Nichtdeterministisch?
 - LLMs sind so
- 👍
 - Reviewer-Agents haben andere, frische Perspektive

Skills

- Automatisierte Workflows
- /plan-and-do
 - Vom Ticket/freier Aufgabe zum PR
 - Teil der Harness, nicht der LLM
- Branch => (PRD =>) Plan => Code => Tests => Review => PR
 - PRD: Product Requirements Document
- Prompts + Skripts/ Programme
 - Prompts: nicht-deterministisch
 - Skripts/Programme: deterministisch
- Global oder Projekt
 - Global: in ~/.claude/skills
 - Projekt: in .claude/skills
- Ruft /review Skill auf
- CLAUDE.md
 - Zum Beispiel: Test-Kommando und Liste der Subagents
- /plan-and-do Write a skill that does...
 - Warum /plan-and-do? Nutzt Subagenten, hat Reviews, macht Git Branch & PR
- /plan-and-do Change skill so...
 - Ich editiere Skills nicht direkt
- Nichtdeterministisch?
 - LLMs sind so
- 👍
 - Skills sind ein gutes Gerüst für Nondeterminismus
- 🙅
 - Nur selten machen sie Mist: /plan-and-do ignoriert dann Prompts und läuft einfach durch

Übungen!

- <https://github.com/atra-consulting/coding-with-ai-lab>
 - ./start.sh
 - start.bat
- DB-Reset

- ./start.sh --reset-db
- start.bat --reset-db
- Aufgaben in tasks/advanced
 - Übungsaufgabe 1: Subagent erstellen
 - Übungsaufgabe 2: Skill erstellen

Zusammenfassung

- Skills automatisieren Workflows
- Subagents schreiben schneller besseren Code

Wege zur Software Factory

- Code-Qualität erhöhen
- Mehr Tests
- Closed Loops
- Automatische Feature Entwicklung mit Feedback Loop
- Vollautomatische Software Factory

1. Code-Qualität erhöhen

- KI-generierte Anwendungs- & Domänen-Doku
- Spec-driven Development
- Skills wie plan-and-do
- Subagents

2. Mehr Tests

- KI-erzeugte, automatische Tests

3. Closed Loops

- Verbesserung von Anwendungs- & Domänen-Doku und Subagenten
 - Bugs, Pull-Request-Kommentare, Support-Chat & Tickets, ...
- Y-Combinator: "Queryable Company"
 - Y-Combinator: Startup-Factory von Microsoft-Mitgründer Paul Allen
 - Siehe <https://www.youtube.com/watch?v=EN7frwQlbKc>
- Make org legible to AI
 - Alles aufnehmen (Meetings & Gespräche)
- Every action creates an artifact than can selfimprove
 - Agenten nutzen, die an der Kommunikation teilhaben
- One-shot internal dashboards for every company ops
 - Für alle Kategorien - Umsatz, Kosten, Engineering, Mitarbeite
- Beispiel: Engineering Agent
- Tickets, Slack/Teams Channels, KundenFeedback, High-LevelPläne, Vertriebs-Anrufe, Transkripte Dailies
 - Darauf – und noch mehr – hat der Agent Zugriff

Automatische Feature Entwicklung mit Feedback Loop

- Arbeitet Tickets ab mit evtl. Nachfrage

Vollautomatische Software Factory

- Arbeitet ohne Nachfrage – schreibt evtl. erst Tickets

Herausforderungen

- Kulturell
- Organisatorisch
- Technisch

Kulturell

- Warum Entwickler?
 - Siehe <https://blog.lmorched.com/2026/03/11/grief-and-the-ai-split/>
- Code schreiben
 - Programmieren als Puzzle lösen, als Kampf mit dem Compiler und dem Linter
 - KI ist furchtbar
 - KI nimmt den Teil der Arbeit weg, der Spass macht
- Computer Dinge tun lassen
 - Programmieren als Weg zum Ziel
 - KI ist wunderbar
- Lösung?
 - Change Management?
 - Mit KI für Bugs & Migrationen starten

Organisatorisch

- Wie Datensammlung erlauben?
 - Jedes Meeting & Gespräch transkribieren
 - Nachrichten, E-Mails und alle Dokumente der KI zugänglich machen
 - Datenschutz, Betriebsrat & Mitarbeiter-Befindlichkeiten
- Wer ist verantwortlich?
 - Wer ist der "Verantwortliche Programmierer", wenn die KI Code schreibt?
 - Was sagt der Regulator dazu in regulierten Branchen?
- "Agile" ist für Menschen
- "AI Agile"?
 - Keine Refinements mehr?
 - KI die Retrospective vorbereiten - was lief gut, was lief schlecht?

Technisch

- Wie Produktivität messen?

- KI ist Investition, die Gewinn bringen muss
- Wie wird das gemessen?
- LLM?
- Harness?
 - US-LLM, die jederzeit abgestellt werden kann? Chinesische Open-Source-LLM? Europäische LLM von Mistral? On Premise oder gehostet?
 - Proprietär, wie Claude Code? Offen, wie GitHub Copilot oder OpenCode?
- Wie gesammelte Daten speichern?
 - Zum Beispiel Meeting-Transkriptionen: Als Text oder mit RAG in Vector-Datenbank?

Diskussion

- KI-Ziel für Organisation?
- Wie dahingelangen?

Zusammenfassung

- Weg
 - Code-Qualität erhöhen
 - Mehr Tests
 - Closed Loops
 - Automatische Feature Entwicklung mit Feedback Loop
 - Vollautomatische Software Factory
- Herausforderungen
 - Kulturell
 - Organisatorisch
 - Technisch

Weg zur Software Factory
